📋 Guide Complet — Configuration Imprimante Thermique MUNBYN RW403B

Version: 1.0
Date: 26 Mars 2026
Système: Debian 12
Contexte: CMDB Inventory — Impression étiquettes QR Code

---

1. Spécifications Techniques MUNBYN RW403B

Caractéristique	Valeur
Type	Thermique directe (sans encre)
Largeur papier	50mm - 80mm
Résolution	203 DPI (8 dots/mm)
Vitesse	150mm/s
Connectivité	Bluetooth 4.0 + USB-C + WiFi
Compatibilité	Windows, macOS, Linux, Android, iOS
Pilotes	Génériques (CUPS) ou aucun (PDF direct)
Commandes	ESC/POS, TSPL, CPCL

---

2. Installation sous Debian 12

2.1 Méthode 1 : Impression PDF (Recommandée ✅)

Aucun pilote requis — Le navigateur gère l'impression.

```
# 1. Mettre à jour le système
sudo apt update && sudo apt upgrade -y

# 2. Installer CUPS (système d'impression Linux)
sudo apt install cups cups-client cups-common -y

# 3. Démarrer et activer CUPS
sudo systemctl enable cups
sudo systemctl start cups

# 4. Ajouter l'utilisateur au groupe lpadmin
sudo usermod -aG lpadmin $USER

# 5. Activer l'interface web CUPS
```

```
sudo cupsctl --remote-any

# 6. Redémarrer CUPS
sudo systemctl restart cups
```

Configuration CUPS :

```
# Accéder à l'interface web
http://localhost:631

# Ou via navigateur
http://<IP_SERVEUR>:631
```

2.2 Méthode 2 : Connexion Bluetooth

```
# 1. Installer outils Bluetooth
sudo apt install bluez bluez-tools blueman -y

# 2. Démarrer service Bluetooth
sudo systemctl enable bluetooth
sudo systemctl start bluetooth

# 3. Vérifier adaptateur Bluetooth
hciconfig -a

# 4. Scanner appareils Bluetooth
bluetoothctl
> scan on
> scan off
> devices
> pair <MAC_ADDRESS>
> trust <MAC_ADDRESS>
> connect <MAC_ADDRESS>
> quit

# 5. Vérifier connexion
bluetoothctl info <MAC_ADDRESS>

# 6. Tester impression
echo "Test" | lp -d <PRINTER_NAME>
```

MAC Address MUNBYN : Généralement commence par `00:11:22:XX:XX:XX`

2.3 Méthode 3 : Connexion USB

```
# 1. Brancher l'imprimante en USB
# 2. Vérifier détection
lsusb

# 3. Vérifier périphérique
ls -l /dev/usb/lp*

# 4. Ajouter permissions
sudo usermod -aG lp $USER

# 5. Tester
echo "Test" > /dev/usb/lp0
```

2.4 Méthode 4 : Connexion WiFi (Network)

```
# 1. Configurer imprimante en WiFi (via app mobile MUNBYN)
# 2. Noter l'adresse IP (ex: 192.168.1.100)

# 3. Ajouter imprimante network dans CUPS
lpadmin -p MUNBYN_RW403B -v socket://192.168.1.100:9100 -E

# 4. Installer driver générique
lpadmin -p MUNBYN_RW403B -m raw

# 5. Définir comme défaut
lpoptions -d MUNBYN_RW403B

# 6. Tester
echo "Test" | lp -d MUNBYN_RW403B
```

---

3. Configuration Imprimante

3.1 Paramètres Recommandés

Paramètre	Valeur	Commande
Type papier	Étiquette thermique	Menu imprimante
Largeur	50mm ou 80mm	Menu imprimante
Hauteur	30mm ou 50mm	Menu imprimante
Densité	10-12 (Medium-High)	Menu imprimante
Vitesse	3 (Normal)	Menu imprimante
Mode	Thermal Direct	Menu imprimante

Paramètre	Valeur	Commande
Interface	Bluetooth/USB/WiFi	Selon connexion

3.2 Calibration Papier

```
1. Éteindre imprimante
2. Maintenir bouton FEED
3. Allumer imprimante (garder FEED)
4. Attendre 2 bips
5. Relâcher FEED
6. Imprimante calibre automatiquement
```

---

4. Backend — Modifications Nécessaires

4.1 `backend/scanner/models.py` (Ajout Modèle Printer)

```python
# backend/scanner/models.py
from django.db import models
import uuid


class Printer(models.Model):
    """
    Configuration des imprimantes thermiques.
    Supporte USB, Bluetooth, WiFi/Ethernet.
    """
    CONNECTION_CHOICES = [
        ('usb', 'USB'),
        ('bluetooth', 'Bluetooth'),
        ('wifi', 'WiFi/Ethernet'),
        ('network', 'Network Printer'),
    ]

    name = models.CharField(max_length=100, default='MUNBYN RW403B')
    connection_type = models.CharField(max_length=20, choices=CONNECTION_CHOICES,
default='bluetooth')

    # Network config
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    port = models.PositiveIntegerField(default=9100)

    # Bluetooth
    mac_address = models.CharField(max_length=17, blank=True, help_text="XX:XX:XX:XX:XX:XX")
    device_path = models.CharField(max_length=255, blank=True)

    # Printer settings
    dpi = models.PositiveIntegerField(default=203)
    speed = models.PositiveIntegerField(default=3, help_text="1-5")
    density = models.PositiveIntegerField(default=10, help_text="1-15")
```

```
    # Paper
    paper_width_mm = models.PositiveIntegerField(default=50)
    paper_height_mm = models.PositiveIntegerField(default=30)

    # Status
    is_active = models.BooleanField(default=True)
    is_default = models.BooleanField(default=False)
    last_used_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-is_default', '-created_at']

    def __str__(self):
        return f"{self.name} ({self.get_connection_type_display()})"

    @property
    def connection_string(self):
        if self.connection_type == 'wifi' and self.ip_address:
            return f"socket://{self.ip_address}:{self.port}"
        elif self.connection_type == 'bluetooth' and self.mac_address:
            return f"bluetooth://{self.mac_address}"
        elif self.connection_type == 'usb':
            return f"usb://lp0"
        return None
```

4.2 `backend/scanner/api/views.py` (Endpoint Print)

```
# backend/scanner/api/views.py
from rest_framework import viewsets, status, decorators
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from django.http import HttpResponse
from django.shortcuts import get_object_or_404
from django.utils import timezone
from scanner.models import Printer, PrintJob, PrintTemplate
from scanner.services.pdf_generator import generate_label_pdf
from inventory.models import Asset
import logging

logger = logging.getLogger(__name__)


class PrinterViewSet(viewsets.ModelViewSet):
    """
    CRUD Imprimantes thermiques.

    list: GET /api/v1/scanner/printers/
    create: POST /api/v1/scanner/printers/
```

```
    retrieve: GET /api/v1/scanner/printers/<id>/
    update: PUT /api/v1/scanner/printers/<id>/
    destroy: DELETE /api/v1/scanner/printers/<id>/
    test: POST /api/v1/scanner/printers/<id>/test/
    """
    queryset = Printer.objects.all()
    serializer_class = PrinterSerializer
    permission_classes = [IsAuthenticated]

    @decorators.action(detail=True, methods=['post'])
    def test(self, request, pk=None):
        """
        Tester l'imprimante avec une étiquette test.

        POST /api/v1/scanner/printers/<id>/test/
        """
        printer = self.get_object()

        try:
            # Générer PDF test
            from io import BytesIO
            from reportlab.pdfgen import canvas
            from reportlab.lib.pagesizes import mm

            buffer = BytesIO()
            width = printer.paper_width_mm * mm
            height = printer.paper_height_mm * mm

            p = canvas.Canvas(buffer, pagesize=(width, height))
            p.setFont("Helvetica-Bold", 12)
            p.drawString(10*mm, height-20*mm, "TEST IMPRESSION")
            p.setFont("Helvetica", 10)
            p.drawString(10*mm, height-30*mm, f"Imprimante: {printer.name}")
            p.drawString(10*mm, height-40*mm, f"Date: {timezone.now().strftime('%d/%m/%Y
%H:%M')}")
            p.drawString(10*mm, height-50*mm, f"Connexion:
{printer.get_connection_type_display()}")
            p.showPage()
            p.save()

            buffer.seek(0)

            # Retourner PDF pour impression navigateur
            response = HttpResponse(buffer.read(), content_type='application/pdf')
            response['Content-Disposition'] = 'attachment; filename="test_print.pdf"'

            printer.last_used_at = timezone.now()
            printer.save(update_fields=['last_used_at'])

            logger.info(f"Test impression réussi: {printer.name}")
            return response

        except Exception as e:
```

```
            logger.error(f"Erreur test impression: {str(e)}")
            return Response(
                {'error': f'Échec test: {str(e)}'},
                status=status.HTTP_400_BAD_REQUEST
            )


@decorators.api_view(['POST'])
@decorators.permission_classes([IsAuthenticated])
def print_labels(request):
    """
    Impression batch d'étiquettes.

    POST /api/v1/scanner/print-labels/
    Body:
    {
        "asset_ids": [152, 153, 154],
        "printer_id": 1,
        "template_id": 1,
        "copies": 1
    }
    """
    asset_ids = request.data.get('asset_ids', [])
    printer_id = request.data.get('printer_id')
    template_id = request.data.get('template_id')
    copies = request.data.get('copies', 1)

    if not asset_ids:
        return Response({'error': 'asset_ids requis'}, status=400)

    # Récupérer assets
    assets = Asset.objects.filter(id__in=asset_ids)
    if not assets.exists():
        return Response({'error': 'Aucun asset trouvé'}, status=404)

    # Template
    template = None
    if template_id:
        template = get_object_or_404(PrintTemplate, id=template_id)
    else:
        template = PrintTemplate.objects.filter(is_default=True).first()

    # Printer
    printer = None
    if printer_id:
        printer = get_object_or_404(Printer, id=printer_id)

    # Générer PDF
    try:
        pdf_buffer = generate_label_pdf(assets, template, printer, copies)

        # Créer PrintJob pour audit
        job = PrintJob.objects.create(
```

```
            created_by=request.user,
            asset_ids=asset_ids,
            template=template,
            printer=printer,
            copies=copies,
            status='completed'
        )

        response = HttpResponse(pdf_buffer.read(), content_type='application/pdf')
        response['Content-Disposition'] = f'attachment;
filename="labels_{timezone.now().strftime("%Y%m%d_%H%M%S")}.pdf"'

        logger.info(f"Impression batch: {len(asset_ids)} assets, Printer: {printer.name if
printer else 'N/A'}")
        return response

    except Exception as e:
        logger.error(f"Erreur impression batch: {str(e)}")
        return Response({'error': str(e)}, status=500)
```

4.3 `backend/scanner/api/serializers.py`

```
# backend/scanner/api/serializers.py
from rest_framework import serializers
from scanner.models import Printer, PrintTemplate, PrintJob


class PrinterSerializer(serializers.ModelSerializer):
    connection_string = serializers.CharField(read_only=True)

    class Meta:
        model = Printer
        fields = '__all__'
        read_only_fields = ['created_at', 'updated_at', 'last_used_at']


class PrintTemplateSerializer(serializers.ModelSerializer):
    class Meta:
        model = PrintTemplate
        fields = '__all__'


class PrintJobSerializer(serializers.ModelSerializer):
    created_by_username = serializers.CharField(source='created_by.username',
read_only=True)
    template_name = serializers.CharField(source='template.name', read_only=True)
    printer_name = serializers.CharField(source='printer.name', read_only=True)

    class Meta:
        model = PrintJob
        fields = '__all__'
```

4.4 `backend/scanner/api/urls.py`

```python
# backend/scanner/api/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import PrinterViewSet, print_labels

router = DefaultRouter()
router.register(r'printers', PrinterViewSet, basename='printer')
router.register(r'templates', PrintTemplateViewSet, basename='print-template')
router.register(r'print-jobs', PrintJobViewSet, basename='print-job')

urlpatterns = [
    path('', include(router.urls)),
    path('print-labels/', print_labels, name='print-labels'),
]
```

5. Frontend — Page Impression

5.1 `templates/admin/scanner/print.html`

```html
{% extends 'admin_base.html' %}
{% load static %}

{% block title %}Impression Étiquettes - CMDB{% endblock %}

{% block extra_css %}
<style>
    .printer-card {
        background: var(--bg-card);
        border: 1px solid rgba(255, 255, 255, 0.1);
        border-radius: 14px;
        padding: 20px;
        margin-bottom: 20px;
    }
    .printer-status {
        display: inline-flex;
        align-items: center;
        gap: 8px;
        padding: 6px 12px;
        border-radius: 20px;
        font-size: 0.85rem;
    }
    .status-online { background: rgba(34, 197, 94, 0.2); color: #22c55e; }
    .status-offline { background: rgba(107, 114, 128, 0.2); color: #6b7280; }
</style>
{% endblock %}

{% block content %}
```

```
<div id="print-app">
    <h2 class="mb-4">🖨️ Impression Étiquettes</h2>

    <!-- Printer Selection -->
    <div class="printer-card">
        <h5 class="mb-3">Sélectionner Imprimante</h5>
        <div class="row g-3">
            <div class="col-md-4" v-for="printer in printers" :key="printer.id">
                <div class="card h-100" :class="{ 'border-primary': selectedPrinter ===
printer.id }"
                     @click="selectedPrinter = printer.id" style="cursor: pointer;">
                    <div class="card-body">
                        <h6 class="card-title">[[ printer.name ]]</h6>
                        <p class="card-text text-muted small">
                            [[ printer.connection_type ]] • [[ printer.paper_width_mm ]]x[[
printer.paper_height_mm ]]mm
                        </p>
                        <span class="printer-status" :class="printer.is_active ? 'status-
online' : 'status-offline'">
                            <span class="dot" style="width:8px;height:8px;border-
radius:50%;background:currentcolor;"></span>
                            [[ printer.is_active ? 'En ligne' : 'Hors ligne' ]]
                        </span>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <!-- Print Options -->
    <div class="printer-card">
        <h5 class="mb-3">Options d'Impression</h5>
        <div class="row g-3">
            <div class="col-md-4">
                <label class="form-label">Template</label>
                <select class="form-select bg-dark border-secondary text-white" v-
model="selectedTemplate">
                    <option v-for="t in templates" :key="t.id" :value="t.id">[[ t.name ]]
</option>
                </select>
            </div>
            <div class="col-md-4">
                <label class="form-label">Copies</label>
                <input type="number" class="form-control bg-dark border-secondary text-
white" v-model="copies" min="1" max="10">
            </div>
            <div class="col-md-4">
                <label class="form-label">Assets sélectionnés</label>
                <input type="text" class="form-control bg-dark border-secondary text-white"
:value="assetIds.length" readonly>
            </div>
        </div>
    </div>
```

```
    <!-- Actions -->
    <div class="d-flex gap-3">
        <button class="btn btn-primary" @click="testPrint" :disabled="!selectedPrinter">
            🧪 Tester Imprimante
        </button>
        <button class="btn btn-success" @click="printLabels" :disabled="!selectedPrinter ||
assetIds.length === 0">
            🖨️ Imprimer Étiquettes
        </button>
        <button class="btn btn-outline-light" @click="downloadPDF">
            📥 Télécharger PDF
        </button>
    </div>
</div>
{% endblock %}

{% block extra_js %}
<script src="{% static 'admin_cmdb/js/print.js' %}"></script>
{% endblock %}
```

5.2 `static/admin_cmdb/js/print.js`

```
// static/admin_cmdb/js/print.js
const { createApp } = window.VueCreateApp || Vue.createApp;

createApp({
    delimiters: ['[[', ']]'],
    data() {
        return {
            printers: [],
            templates: [],
            assetIds: [],
            selectedPrinter: null,
            selectedTemplate: null,
            copies: 1,
            printing: false
        }
    },
    mounted() {
        this.fetchPrinters();
        this.fetchTemplates();
        this.getAssetIdsFromURL();
    },
    methods: {
        async fetchPrinters() {
            try {
                const res = await window.apiClient.get('/scanner/printers/');
                this.printers = res.data;
                if (this.printers.length > 0) {
```

```
                    this.selectedPrinter = this.printers.find(p => p.is_default)?.id ||
this.printers[0].id;
                }
            } catch (error) {
                console.error('Erreur fetch printers:', error);
            }
        },
        async fetchTemplates() {
            try {
                const res = await window.apiClient.get('/scanner/templates/');
                this.templates = res.data;
                if (this.templates.length > 0) {
                    this.selectedTemplate = this.templates.find(t => t.is_default)?.id ||
this.templates[0].id;
                }
            } catch (error) {
                console.error('Erreur fetch templates:', error);
            }
        },
        getAssetIdsFromURL() {
            const params = new URLSearchParams(window.location.search);
            const ids = params.get('assets');
            if (ids) {
                this.assetIds = ids.split(',').map(id => parseInt(id));
            }
        },
        async testPrint() {
            if (!this.selectedPrinter) return;

            try {
                const res = await
window.apiClient.post(`/scanner/printers/${this.selectedPrinter}/test/`, {}, {
                    responseType: 'blob'
                });

                const url = window.URL.createObjectURL(res);
                const link = document.createElement('a');
                link.href = url;
                link.download = 'test_print.pdf';
                link.click();
                window.URL.revokeObjectURL(url);

                alert('✅ Test impression lancé');
            } catch (error) {
                alert('❌ Erreur test impression: ' + error.message);
            }
        },
        async printLabels() {
            if (!this.selectedPrinter || this.assetIds.length === 0) return;

            this.printing = true;

            try {
```

```
                const res = await window.apiClient.post('/scanner/print-labels/', {
                    asset_ids: this.assetIds,
                    printer_id: this.selectedPrinter,
                    template_id: this.selectedTemplate,
                    copies: this.copies
                }, {
                    responseType: 'blob'
                });

                const url = window.URL.createObjectURL(res);
                const link = document.createElement('a');
                link.href = url;
                link.download = `labels_${Date.now()}.pdf`;
                link.click();
                window.URL.revokeObjectURL(url);

                alert('✅ Impression lancée');
            } catch (error) {
                alert('❌ Erreur impression: ' + error.message);
            } finally {
                this.printing = false;
            }
        },
        async downloadPDF() {
            await this.printLabels();
        }
    }
}).mount('#print-app');
```

6. Procédure Complète d'Installation

6.1 Checklist Installation

Étape	Commande	Statut
1. Installer CUPS	`sudo apt install cups -y`	⬜
2. Activer Bluetooth	`sudo systemctl start bluetooth`	⬜
3. Appairer imprimante	`bluetoothctl` → pair/connect	⬜
4. Configurer CUPS	`http://localhost:631`	⬜
5. Tester impression	`echo "Test" | lp -d MUNBYN`	⬜
6. Créer Printer DB	Django Admin → Scanner → Printers	⬜
7. Tester API	POST `/scanner/printers/<id>/test/`	⬜
8. Imprimer étiquette	Frontend → Print page	⬜

6.2 Commandes de Test

```
# 1. Vérifier imprimante CUPS
lpstat -p

# 2. Vérifier file d'attente
lpstat -o

# 3. Tester impression directe
echo "Test" | lp -d MUNBYN_RW403B

# 4. Annuler job
cancel <job_id>

# 5. Logs CUPS
tail -f /var/log/cups/error_log

# 6. Permissions
ls -l /dev/usb/lp*
ls -l /dev/rfcomm*
```

---

7. Dépannage

7.1 Problèmes Courants

Problème	Solution
Imprimante non détectée	`sudo systemctl restart bluetooth`
Appairage échoue	`bluetoothctl remove <MAC>` puis réappairer
Impression blanche	Augmenter density (10→12)
Étiquette mal alignée	Calibrer papier (voir section 3.2)
PDF trop grand	Vérifier template (50x30mm vs 80x50mm)
Erreur permissions	`sudo usermod -aG lp $USER` puis reboot

7.2 Logs à Vérifier

```
# CUPS errors
sudo tail -f /var/log/cups/error_log

# Bluetooth
sudo journalctl -u bluetooth -f

# Django
tail -f /var/log/django/cmdb.log
```

---

8. Résumé — Modifications Backend

Fichier	Modification	Nécessaire ?
`scanner/models.py`	Ajouter modèle `Printer`	✅ Oui
`scanner/api/views.py`	Ajouter `PrinterViewSet` + `print_labels`	✅ Oui
`scanner/api/serializers.py`	Ajouter `PrinterSerializer`	✅ Oui
`scanner/api/urls.py`	Router printers + print-labels	✅ Oui
`templates/admin/scanner/print.html`	Page impression	✅ Oui
`static/admin_cmdb/js/print.js`	Vue app impression	✅ Oui

✅ Conclusion

Question	Réponse
Pilote dédié requis ?	❌ Non — PDF via navigateur suffit
CUPS nécessaire ?	✅ Oui (gestion impressions Linux)
Bluetooth fonctionne ?	✅ Oui (via `bluetoothctl`)
Backend à modifier ?	✅ Oui (modèles + endpoints)
Frontend à modifier ?	✅ Oui (page impression dédiée)
Temps installation ?	⏱️ 30-60 minutes

Cette configuration est production-ready pour MUNBYN RW403B sous Debian 12 ! 🎉

📋 Manuel de Configuration — Imprimante MUNBYN RW403B

Système: Debian 12 (Bookworm)
Version: 1.0 — Mars 2026
Connexion: USB (`/dev/usb/lp0`)
Contexte: CMDB Inventory — Impression étiquettes QR Code

---

1. 🖨️ Spécifications Techniques MUNBYN RW403B

Caractéristique	Valeur
Type	Thermique directe (sans encre)

Caractéristique	Valeur
Largeur papier	50mm - 80mm
Résolution	203 DPI (8 dots/mm)
Vitesse	150mm/s
Connectivité	USB-C + Bluetooth 4.0 + WiFi
Compatibilité	Windows, macOS, Linux, Android, iOS
Langages	ESC/POS, TSPL, CPCL
Pilotes Linux	Génériques (CUPS)
Périphérique USB	`/dev/usb/lp0`

---

2. ✅ Compatibilité POS & Django

2.1 Question: Compatible POS pour Django?

Aspect	Réponse	Détails
ESC/POS	✅ **OUI**	Langage standard imprimantes thermiques
CUPS Linux	✅ **OUI**	Pilote générique "Raw" ou "Thermal Printer"
Accès Django	✅ **OUI**	Via CUPS API ou génération PDF directe
Python Libraries	✅ **OUI**	`python-escpos`, `pycups`, `reportlab`
USB Direct	✅ **OUI**	Accès via `/dev/usb/lp0` avec permissions
Network	✅ **OUI**	Socket TCP/IP port 9100

2.2 Architecture d'Accès Django

```
┌──────────────────────────────────────────────────────────────────┐
│              ARCHITECTURE ACCÈS IMPRIMANTE                       │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ┌──────────────┐          ┌──────────────┐                     │
│  │   Django     │          │    CUPS      │                     │
│  │   Backend    │──────────→│   Service    │                     │
│  │              │   PDF    │              │                     │
│  └──────────────┘          └──────────────┘                     │
│                                   │                              │
│                              USB/BT/WiFi                         │
│                                   │                              │
│                                   ▼                              │
│                          ┌──────────────┐                        │
```

```
│                        │  MUNBYN     │                          │
│                        │  RW403B     │                          │
│                        └─────────────┘                          │
│                                                                 │
│  MÉTHODES D'ACCÈS:                                              │
│  ─────────────────                                              │
│  1. PDF → CUPS → Imprimante (Recommandé ✅)                    │
│  2. ESC/POS → python-escpos → USB (Avancé)                     │
│  3. Raw TCP → Socket 9100 (Network)                            │
│  4. Device file → /dev/usb/lp0 (Direct)                        │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

---

3. 📦 Installation Debian 12

3.1 Prérequis Système

```
# Mettre à jour le système
sudo apt update && sudo apt upgrade -y

# Installer CUPS (Common Unix Printing System)
sudo apt install cups cups-client cups-common cups-bsd -y

# Installer outils Bluetooth (si besoin)
sudo apt install bluez bluez-tools blueman -y

# Installer dépendances Python
sudo apt install python3-pip python3-dev libcups2-dev -y

# Installer libraries Python pour impression
pip3 install pycups reportlab python-escpos pillow
```

3.2 Activer et Démarrer Services

```
# Activer CUPS au démarrage
sudo systemctl enable cups
sudo systemctl start cups

# Vérifier statut CUPS
sudo systemctl status cups

# Activer Bluetooth (si imprimante Bluetooth)
sudo systemctl enable bluetooth
sudo systemctl start bluetooth

# Vérifier statut Bluetooth
sudo systemctl status bluetooth
```

3.3 Configuration Utilisateurs

```
# Ajouter l'utilisateur au groupe lpadmin (administration imprimantes)
sudo usermod -aG lpadmin $USER

# Ajouter l'utilisateur au groupe lp (impression)
sudo usermod -aG lp $USER

# Ajouter l'utilisateur www-data (Django) au groupe lp
sudo usermod -aG lp www-data

# Appliquer les changements (déconnexion/reconnexion requise)
newgrp lp
newgrp lpadmin
```

---

4. 🔌 Connexion USB Configuration

4.1 Brancher et Détecter l'Imprimante

```
# 1. Brancher l'imprimante en USB
# 2. Vérifier détection système
lsusb

# Sortie attendue:
# Bus 001 Device 005: ID 0483:5740 STMicroelectronics MUNBYN Printer

# 3. Vérifier périphérique USB parallel
ls -l /dev/usb/lp*

# Sortie attendue:
# crw-rw---- 1 root lp 180, 0 Mar 26 10:00 /dev/usb/lp0

# 4. Vérifier permissions
ls -l /dev/usb/lp0
```

4.2 Corriger Permissions USB

```
# Si permissions incorrectes, créer règle udev
sudo nano /etc/udev/rules.d/99-munbyn-printer.rules

# Ajouter cette ligne:
KERNEL=="lp[0-9]*", SUBSYSTEM=="usb", ATTRS{idVendor}=="0483", MODE="0666", GROUP="lp"

# Recharger règles udev
sudo udevadm control --reload-rules
sudo udevadm trigger

# Vérifier permissions (doit être 0666 ou 0660 avec groupe lp)
ls -l /dev/usb/lp0
# crw-rw-rw- 1 root lp 180, 0 Mar 26 10:00 /dev/usb/lp0
```

4.3 Tester Communication Directe

```
# Tester envoi direct de données à l'imprimante
echo "Test impression directe" | sudo tee /dev/usb/lp0

# Ou avec hexdump pour vérifier
echo "TEST" | sudo hexdump -C > /dev/usb/lp0

# Si erreur "Permission denied", vérifier permissions ci-dessus
```

---

5. ⚙️ Configuration CUPS

5.1 Activer Interface Web CUPS

```
# Éditer configuration CUPS
sudo nano /etc/cups/cupsd.conf

# Modifier/ajouter ces lignes:
# Listen localhost:631
Listen 631
Listen /var/run/cups/cups.sock

# <Location />
#   Order allow,deny
#   Allow all
# </Location>

# <Location /admin>
#   Order allow,deny
#   Allow all
# </Location>

# <Location /admin/conf>
```

```
#   AuthType Default
#   Require user @SYSTEM
#   Order allow,deny
#   Allow all
# </Location>

# Redémarrer CUPS
sudo systemctl restart cups
```

5.2 Accéder à l'Interface CUPS

```
# Ouvrir navigateur
http://localhost:631

# Ou depuis serveur distant
http://<IP_SERVEUR>:631

# Identifiants:
# Username: votre_user_linux
# Password: mot_de_passe_linux
```

5.3 Ajouter Imprimante dans CUPS

Via Interface Web:

```
1. Aller sur: http://localhost:631
2. Cliquer: "Administration"
3. Cliquer: "Add Printer"
4. Sélectionner: "MUNBYN RW403B" (ou "USB Printer")
5. Cliquer: "Continue"
6. Nom: MUNBYN_RW403B
7. Description: Imprimante Étiquettes QR - Atelier
8. Location: Atelier IT
9. Cliquer: "Add Printer"
10. Modèle: "Raw Queue" (recommandé pour PDF)
    OU "Thermal Printer 80mm" (pour ESC/POS)
11. Cliquer: "Add Printer"
12. Définir comme imprimante par défaut (optionnel)
```

Via Ligne de Commande:

```
# Lister imprimantes détectées
lpinfo -v

# Sortie attendue:
# direct usb://MUNBYN/RW403B?serial=1234567890
# direct socket://192.168.1.100:9100
# direct parallel:/dev/usb/lp0

# Ajouter imprimante USB
sudo lpadmin -p MUNBYN_RW403B \
```

```
    -v usb://MUNBYN/RW403B \
    -m raw \
    -E

# OU avec device path direct
sudo lpadmin -p MUNBYN_RW403B \
    -v parallel:/dev/usb/lp0 \
    -m raw \
    -E

# Activer imprimante
sudo cupsenable MUNBYN_RW403B
sudo cupsaccept MUNBYN_RW403B

# Définir comme par défaut
sudo lpoptions -d MUNBYN_RW403B

# Vérifier configuration
lpstat -p MUNBYN_RW403B
lpstat -v
```

---

6. 🧪 Tests d'Impression

6.1 Test Basique

```
# Test simple avec texte
echo "Test impression MUNBYN" | lp -d MUNBYN_RW403B

# Vérifier file d'attente
lpstat -o

# Annuler job si besoin
cancel <job_id>
```

6.2 Test PDF (Recommandé pour CMDB)

```
# Créer PDF test avec Python
python3 << 'EOF'
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import mm
from io import BytesIO

buffer = BytesIO()
p = canvas.Canvas(buffer, pagesize=(50*mm, 30*mm))
p.setFont("Helvetica-Bold", 12)
p.drawString(5*mm, 25*mm, "TEST QR")
p.setFont("Helvetica", 8)
p.drawString(5*mm, 15*mm, "MUNBYN RW403B")
p.drawString(5*mm, 10*mm, "Debian 12 + CUPS")
p.showPage()
```

```
p.save()

with open('/tmp/test_label.pdf', 'wb') as f:
    f.write(buffer.getvalue())

print("PDF créé: /tmp/test_label.pdf")
EOF

# Imprimer PDF
lp -d MUNBYN_RW403B /tmp/test_label.pdf

# OU avec options spécifiques
lp -d MUNBYN_RW403B \
    -o media=50x30mm \
    -o fit-to-page \
    -o landscape \
    /tmp/test_label.pdf
```

6.3 Test ESC/POS (Si mode raw)

```
# Installer python-escpos
pip3 install python-escpos

# Test script Python
python3 << 'EOF'
from escpos.printer import Usb

# Trouver vendor/product ID avec lsusb
# Ex: Bus 001 Device 005: ID 0483:5740
printer = Usb(0x0483, 0x5740)

printer.text("Test MUNBYN RW403B\n")
printer.text("Debian 12 + ESC/POS\n")
printer.text("==================\n\n")
printer.qr("https://cmdb.example.com/asset/210")
printer.cut()
EOF
```

---

7. 🔧 Configuration Imprimante

7.1 Paramètres Recommandés CUPS

```
# Définir options par défaut
lpoptions -p MUNBYN_RW403B -o media=50x30mm
lpoptions -p MUNBYN_RW403B -o fit-to-page
lpoptions -p MUNBYN_RW403B -o cpi=17
lpoptions -p MUNBYN_RW403B -o lpi=8

# Vérifier options disponibles
lpoptions -p MUNBYN_RW403B -l
```

7.2 Calibration Papier (Manuel)

```
┌──────────────────────────────────────────────────────────────────────┐
│              CALIBRATION MANUELLE MUNBYN                             │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  1. Éteindre imprimante (switch OFF)                                 │
│                                                                      │
│  2. Maintenir bouton FEED appuyé                                     │
│                                                                      │
│  3. Allumer imprimante (switch ON)                                   │
│     (garder FEED appuyé)                                             │
│                                                                      │
│  4. Attendre 2 bips sonores                                          │
│                                                                      │
│  5. Relâcher bouton FEED                                             │
│                                                                      │
│  6. Imprimante calibre automatiquement                               │
│     (sortie 1-2 étiquettes vierges)                                  │
│                                                                      │
│  7. Appuyer 1x sur FEED pour vérifier                                │
│     (doit sortir exactement 1 étiquette)                             │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

7.3 Configuration via Boutons

Action	Résultat
1x FEED	Sortie 1 étiquette
2x FEED	Calibration auto
3x FEED	Test impression interne
FEED + Power	Reset usine

---

8. 🐍 Intégration Django

8.1 Modèle Printer (Déjà dans scanner/models.py)

```python
# backend/scanner/models.py
class Printer(models.Model):
    CONNECTION_CHOICES = [
        ('usb', 'USB'),
        ('bluetooth', 'Bluetooth'),
        ('wifi', 'WiFi/Ethernet'),
        ('network', 'Network Printer'),
    ]
```

```
    name = models.CharField(max_length=100, default='MUNBYN RW403B')
    connection_type = models.CharField(max_length=20, choices=CONNECTION_CHOICES,
default='usb')
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    port = models.PositiveIntegerField(default=9100)
    mac_address = models.CharField(max_length=17, blank=True)
    device_path = models.CharField(max_length=255, blank=True, default='/dev/usb/lp0')
    dpi = models.PositiveIntegerField(default=203)
    speed = models.PositiveIntegerField(default=3)
    density = models.PositiveIntegerField(default=10)
    paper_width_mm = models.PositiveIntegerField(default=50)
    paper_height_mm = models.PositiveIntegerField(default=30)
    is_active = models.BooleanField(default=True)
    is_default = models.BooleanField(default=False)
    last_used_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

8.2 Service d'Impression Django

```
# backend/scanner/services/printer_service.py
import subprocess
import logging
from django.conf import settings
from pathlib import Path

logger = logging.getLogger(__name__)


class PrinterService:
    """Service d'impression via CUPS pour Django."""

    def __init__(self, printer_name='MUNBYN_RW403B'):
        self.printer_name = printer_name

    def print_pdf(self, pdf_path, copies=1, options=None):
        """
        Imprimer un PDF via CUPS.

        Args:
            pdf_path: Chemin du fichier PDF
            copies: Nombre de copies
            options: Dict options CUPS

        Returns:
            bool: True si succès
        """
        cmd = ['lp', '-d', self.printer_name, '-n', str(copies)]

        if options:
            for key, value in options.items():
                cmd.extend(['-o', f'{key}={value}'])
```

```
        cmd.append(pdf_path)

        try:
            result = subprocess.run(cmd, capture_output=True, text=True, timeout=30)

            if result.returncode == 0:
                logger.info(f"Impression réussie: {pdf_path}")
                return True
            else:
                logger.error(f"Erreur impression: {result.stderr}")
                return False

        except subprocess.TimeoutExpired:
            logger.error("Timeout impression")
            return False
        except Exception as e:
            logger.error(f"Exception impression: {str(e)}")
            return False

    def print_pdf_buffer(self, pdf_buffer, copies=1, options=None):
        """
        Imprimer depuis buffer PDF en mémoire.

        Args:
            pdf_buffer: BytesIO buffer PDF
            copies: Nombre de copies
            options: Dict options CUPS

        Returns:
            bool: True si succès
        """
        import tempfile

        with tempfile.NamedTemporaryFile(suffix='.pdf', delete=False) as tmp:
            tmp.write(pdf_buffer.getvalue())
            tmp_path = tmp.name

        try:
            success = self.print_pdf(tmp_path, copies, options)
            Path(tmp_path).unlink()  # Cleanup
            return success
        except Exception as e:
            logger.error(f"Erreur cleanup: {str(e)}")
            return False

    def get_printer_status(self):
        """Vérifier statut imprimante."""
        cmd = ['lpstat', '-p', self.printer_name, '-l']

        try:
            result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
            return {
```

```
                'available': 'idle' in result.stdout.lower(),
                'status': result.stdout.strip()
            }
        except Exception as e:
            return {
                'available': False,
                'status': f'Error: {str(e)}'
            }

    def get_queue(self):
        """Récupérer jobs en attente."""
        cmd = ['lpstat', '-o', self.printer_name]

        try:
            result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
            jobs = []
            for line in result.stdout.strip().split('\n'):
                if line:
                    parts = line.split()
                    if len(parts) >= 3:
                        jobs.append({
                            'job_id': parts[1],
                            'user': parts[2],
                            'size': parts[3] if len(parts) > 3 else 'N/A'
                        })
            return jobs
        except Exception as e:
            return []

    def cancel_job(self, job_id):
        """Annuler un job d'impression."""
        cmd = ['cancel', job_id]

        try:
            result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
            return result.returncode == 0
        except Exception as e:
            logger.error(f"Erreur annulation: {str(e)}")
            return False


# Usage dans une view Django
def print_label_view(request, asset_id):
    from scanner.services.pdf_generator import generate_label_pdf
    from scanner.services.printer_service import PrinterService

    asset = get_object_or_404(Asset, id=asset_id)
    pdf_buffer = generate_label_pdf([asset], format='50x30')

    printer = PrinterService('MUNBYN_RW403B')
    success = printer.print_pdf_buffer(pdf_buffer, copies=1)

    if success:
```

```
        return JsonResponse({'status': 'ok', 'message': 'Impression lancée'})
    else:
        return JsonResponse({'status': 'error', 'message': 'Échec impression'}, status=500)
```

8.3 API Endpoint Impression

```
# backend/scanner/api/views.py
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from scanner.services.printer_service import PrinterService
from scanner.services.pdf_generator import generate_label_pdf
from inventory.models import Asset


@api_view(['POST'])
@permission_classes([IsAuthenticated])
def print_to_printer(request, asset_id):
    """
    Imprimer étiquette directement sur imprimante configurée.

    POST /api/v1/scanner/print-to/<asset_id>/
    Body: {
        "copies": 1,
        "printer_name": "MUNBYN_RW403B"
    }
    """
    try:
        asset = Asset.objects.get(id=asset_id)
    except Asset.DoesNotExist:
        return Response({'error': 'Asset introuvable'}, status=404)

    copies = request.data.get('copies', 1)
    printer_name = request.data.get('printer_name', 'MUNBYN_RW403B')

    # Générer PDF
    pdf_buffer = generate_label_pdf([asset], format='50x30', copies=copies)

    # Imprimer via CUPS
    printer = PrinterService(printer_name)
    success = printer.print_pdf_buffer(pdf_buffer, copies=1)

    if success:
        # Enregistrer PrintLog
        from scanner.models import PrintLog
        PrintLog.objects.create(
            asset=asset,
            printed_by=request.user,
            printer_name=printer_name,
            template_name='Standard 50x30',
            ip_address=request.META.get('REMOTE_ADDR'),
```

```
            user_agent=request.META.get('HTTP_USER_AGENT')
        )

        return Response({'status': 'ok', 'message': 'Impression lancée'})
    else:
        return Response({'status': 'error', 'message': 'Échec impression'}, status=500)
```

---

9. 🐛 Dépannage

9.1 Problèmes Courants

Problème	Cause	Solution
Permission denied /dev/usb/lp0	Permissions incorrectes	Rule udev + groupe lp
Imprimante non détectée	Câble USB défectueux	Changer câble/port USB
Impression blanche	Density trop faible	Augmenter à 12-14
Étiquette mal alignée	Calibration requise	Procédure calibration (section 7.2)
CUPS ne voit pas imprimante	Service non démarré	`sudo systemctl restart cups`
Job reste en attente	File d'attente bloquée	`sudo cancel -a` puis restart cups
Django ne peut pas imprimer	www-data pas dans groupe lp	`sudo usermod -aG lp www-data`

9.2 Commandes de Diagnostic

```
# Vérifier service CUPS
sudo systemctl status cups

# Redémarrer CUPS
sudo systemctl restart cups

# Lister imprimantes
lpstat -p

# Lister imprimantes disponibles
lpinfo -v

# Vérifier file d'attente
lpstat -o

# Annuler tous jobs
sudo cancel -a
```

```
# Logs CUPS
sudo tail -f /var/log/cups/error_log

# Logs système USB
sudo dmesg | grep -i usb

# Vérifier permissions device
ls -l /dev/usb/lp0

# Test communication directe
echo "TEST" | sudo tee /dev/usb/lp0
```

9.3 Logs à Consulter

```
# Erreurs CUPS
sudo tail -100 /var/log/cups/error_log

# Accès CUPS
sudo tail -100 /var/log/cups/access_log

# Messages système
sudo journalctl -u cups -f

# USB events
sudo journalctl -f | grep -i usb
```

---

10. ✅ Checklist Finale

Étape	Commande	Statut
1. Installer CUPS	`sudo apt install cups -y`	⬜
2. Activer service	`sudo systemctl enable cups`	⬜
3. Ajouter utilisateur à lp	`sudo usermod -aG lp $USER`	⬜
4. Ajouter www-data à lp	`sudo usermod -aG lp www-data`	⬜
5. Brancher imprimante USB	Connecter MUNBYN RW403B	⬜
6. Vérifier device	`ls -l /dev/usb/lp0`	⬜
7. Configurer udev rules	`/etc/udev/rules.d/99-munbyn-printer.rules`	⬜
8. Ajouter dans CUPS	`lpadmin -p MUNBYN_RW403B -v parallel:/dev/usb/lp0 -m raw -E`	⬜
9. Tester impression	`echo "Test" | lp -d MUNBYN_RW403B`	⬜
10. Calibration papier	Procédure manuelle (section 7.2)	⬜

Étape	Commande	Statut
11. Configurer Django	Modèle Printer + PrinterService	⬜
12. Test PDF Django	`/api/v1/scanner/print-to/<id>/`	⬜

11. 📊 Résumé Compatibilité

Question	Réponse
Compatible POS (ESC/POS)?	✅ **OUI**
Compatible CUPS Linux?	✅ **OUI**
Compatible Django?	✅ **OUI** (via CUPS ou python-escpos)
USB direct possible?	✅ **OUI** (`/dev/usb/lp0`)
PDF supporté?	✅ **OUI** (recommandé pour CMDB)
Bluetooth supporté?	✅ **OUI** (appairage requis)
WiFi supporté?	✅ **OUI** (configuration IP)
Driver dédié requis?	❌ **NON** (driver générique suffit)

12. 📞 Support & Ressources

Ressource	URL
MUNBYN Documentation	https://munbyn.com/support
CUPS Documentation	https://www.cups.org/documentation.html
python-escpos	https://python-escpos.readthedocs.io/
ReportLab PDF	https://www.reportlab.com/docs/
Debian Printing	https://wiki.debian.org/SystemPrinting

Cette configuration est production-ready pour MUNBYN RW403B sous Debian 12 avec Django CMDB Inventory! 🎉